FOUNDATIONS FOR A FAST INTEGER MULTIPLIER ON MATHLIB'S TURING MACHINES

DAN BROWN*

Abstract. We reduce the Mathlib statement `MathlibTM2.Statement` (integer multiplication in time $O(n(\log n)^{1-\kappa})$ on `Turing.FinTM2`) to the existence of a single structured host program that multiplies long inputs within that time. Toward that program we prove in Lean: the complete numeric algorithm of the paper as one concrete function whose rounded output is the product for every large input, with pairwise coprime moduli in place of primes; the simultaneous butterfly layer as an exact computation on Gaussian integers; the round decomposition and error bound of the synthetic transforms; the solution of the layer's cost recurrence; and a library of host machines with exact step counts, including the network gate, sign, leaf, packed-addition and node-body passes of the layer and the input and output stages of the multiplier. All theorems depend only on `propext`, `Classical.choice` and `Quot.sound`.

1. The reduction

Everything lives under `IntegerMultBounds.MathlibTM2` and builds on the Mathlib statement and the structured stack language `Stack.lean` of the preceding contribution. A *host program* is a procedure table in that language over the Mathlib input and output stacks and a bank of register stacks. The predicate `HostMulWithin` says that on input $x\#y$ the program halts in the exact final configuration required by `TM2ComputableInTime` within the given time.

Theorem 1 (`Fast.statement_of_dispatch`)**.** *If a host program satisfies* `HostMulWithin` *on every input of encoded length at least n_0, with time $O(n(\log n)^{1-\kappa})$ for some $\kappa > 0$, then* `MathlibTM2.Statement` *holds.*

The proof wraps the program in a dispatcher that reads up to n_0 input symbols into its finite control. A shorter input is answered from a finite table; otherwise the symbols are pushed back and the fast program runs.

2. The numeric algorithm

`Mult.Pipeline.pipelineCoeffs n x y` is the paper's computation with every ingredient fixed: the parameters $b, p, d, K, T, r, \ell, t_i, s_i, \alpha, \gamma$ of `Mult.Sizes`; the CRT reindexing Φ of `Mult.CRT`; the line maps $\widetilde{A}$, $\widetilde{B}_0$ with the vendored weight tables; the chirp table; the computed complex convolution through twisted synthetic transforms; pointwise products; and the final scaling and rounding.

Theorem 2 (`Mult.Pipeline.pipeline_correct`)**.** *For all large n and all $x, y < 2^n$,*

$$\mathrm{evalBase}\ (2^b)\ (\mathit{pipelineCoeffs}\ n\ x\ y) = xy.$$

The chain behind it is the vendored `ExactRecovery.exact_product` together with new ingredients.

Date: October 11, 2026.

*Author of *The Da Vinci Code*.

Coprime moduli. No prime number theorem is needed. Let N be the product of the odd numbers below $4d$, take group A moduli $Nx + 1$ near $r/2$ for consecutive even x, and group B moduli $Ny + 2$ near r for consecutive odd y with $y \approx 2x$. A common divisor of two moduli has no prime factor below $4d$ and divides one of $y - 2x$, $x - x'$, $y - y'$, all nonzero and at most $4d$; so the moduli are pairwise coprime, and for $r \geq 16d^2N + 8d$ they lie in the required intervals (`Mult.Moduli.src_good`).

Errors. The source transform $\widetilde{F} = 2^{\gamma}\,\widetilde{B}_0\,\widetilde{C}\,\widetilde{A}$ is within $9 \cdot 2^{\gamma} T \log_2 T$ grid units of $RF_s\Phi^{-1}$ on the half ball (`Mult.SourceErr`), including the weighted chirp, restricted-domain compositions and the opposite transform by conjugation. The computed complex convolution is within $E_{\mathrm{ring}} + 9$ units (`Mult.ComplexConv`), and the synthetic convolution of the twisted records has $E_{\mathrm{ring}} = \sqrt{2}\,M(3\ell + 1) < 5T \log_2 T$ (`Mult.SynthErr`). The forward and opposite synthetic transforms are exactly ℓ rounds of normalized butterflies on one selected bit per axis followed by per-record twiddles $y^{\mp E}$ (`Mult.Rounds`).

A cheaper $\widetilde{B}_0$. The vendored tabled $\widetilde{B}_0$ runs p Neumann steps. With $\|N - I\| \leq 2^{-\alpha^2\theta}$ (`opNorm_offDiagCLM_le_rpow`) the pipeline uses α^2 steps and an off-diagonal window of $\lfloor\sqrt{p}\rfloor/(2\alpha) + 1$ terms, with the same error conclusion (`Mult.FastB0.resampB0NumK_spec`): $O(s\,\alpha\sqrt{p})$ term products per line.

3. The simultaneous layer on Gaussian integers

The machine stores Gaussian integers modulo 2^w. Each network gate is doubled (its coefficients are halves of integers, `program_gate_coeff`) and doubles the wires it does not write; a child returns 2^{T_c} times its result and the other wires are scaled by 2^{T_c}. With this uniform exponent every step is a ring operation, so intermediate overflow is harmless.

Theorem 3 (`Layer.nodeM_correct`, `Layer.layerZ_correct`)**.** *The scaled node, and by induction the recursive layer `layerZ` on $W^j R_0$ rows, computes 2^E times the kernel C on every active bit.*

Theorem 4 (`Layer.topZ_correct`)**.** *The full layer pipeline (phase S, kernels on the top chunks, rows and padding, base-m pieces through `layerZ`, phase S, the unit $(1 - i)^D 2^{-\lfloor D/2\rfloor}$, truncating shift) returns the componentwise truncation toward zero of the normalized butterflies H_0 on the selected bits.*

The cost recurrence of the paper is solved in `Layer.Recurrence.layer_cost_bound`: with branching $s/W < m^{\sigma}$ (`ComplexRank25.branching_bound`) and the interchange exponent τ (`BinaryInterchangeRun.uniform_bo`) the normalized cost of a layer is $O(d^{\lambda'})$.

4. Host machines

Every machine below comes with an `_exec` theorem giving its exact final register contents and an explicit step bound.

- *Toolkit* (`Host`): foreign calls running any vendored `Machine.Program` with a `HoareTime` contract on host registers; streams, block routing, word automata; index-driven routing and merging by address codes; stable radix sort; exact binary arithmetic (addition, comparison, multiplication, division); the vendored fast interchange of address chunks; the vendored Schönhage–Strassen multiplier (`ssMul_exec_log`).
- *Layer passes* (`Layer`, `Node`): Gaussian linear automata, scaling by 2^s, the doubled gate on a bank of wire registers, address-dependent phases and signs, the leaf kernels, the packed slot-XOR passes (controlled rotations, field swaps through the fast interchange, key attachment, exact repair by extraction, radix sort and reinsertion, top-bit fixup), unary parameter and mask utilities, and the node body (`LayerMachine.nodeBody_exec_layerZ`) with abstract factor programs.
- *Multiplier stages* (`Fast`): reading the input and writing the scaled digit records; carry propagation and the Mathlib-encoded output.

5. Status

The fast host program is not yet assembled. Its remaining components are the factor programs and the recursion of the layer machine, the synthetic transform machine, the linewise resampling machines, the table machines, the CRT reindexing machine, the pointwise and scaling stages, and the total cost bound.

6. Verification

`lake build` of the library passes, `AxiomAudit.lean` passes, and `FastPathCheck.lean` asserts the axioms of the main theorems above.

Email address: jdb1729@gmail.com